# Proyecto 1 — Entrega 1

Despliegue manual de un microservicio sobre un servicio de Plataforma como Servicio (PaaS)

**Cloud Computing**

Universidad de los Andes

**Jesús Daniel Lorduy**

j.lorduy@uniandes.edu.co

18 de octubre de 2026

| Integrante | Correo |
|---|---|
| Jesús Daniel Lorduy | j.lorduy@uniandes.edu.co |

# Enlaces de la entrega

| Recurso | Enlace |
|---|---|
| Repositorio GitHub | https://github.com/LorduyANDES/blacklist-microservice |
| Aplicacion desplegada | http://blacklist-env.eba-zgnk2x3x.us-east-2.elasticbeanstalk.com |
| Documentacion publicada de Postman | https://documenter.getpostman.com/view/59043927/2sBYHSy11e |
| Video de sustentacion (max. 10 min) | Pendiente: insertar el enlace al grabar |

# 1. Alcance de la solucion

Se desarrollo y desplego un microservicio REST que gestiona la lista negra global de emails de una compania multinacional. El objetivo es que cientos de sistemas internos puedan consultar si un email esta en la lista negra global, asi como agregar emails a ella, desde un servicio centralizado.

El microservicio expone dos operaciones de negocio y un health check, todas sobre un unico servicio, y persiste la informacion en una base de datos relacional administrada (AWS RDS PostgreSQL).

# 2. Stack tecnologico

Se utilizo el stack exigido por el enunciado:

| Componente | Version | Uso en el proyecto |
|---|---|---|
| Python | 3.13 | Lenguaje de programacion |
| Flask | 3.1.0 | Micro framework web |
| Flask-SQLAlchemy | 3.1.1 | ORM sobre PostgreSQL |
| Flask-RESTful | 0.3.10 | Recursos REST orientados a objetos |
| Flask-Marshmallow | 1.3.0 | Validacion y serializacion |
| Flask-JWT-Extended | 4.7.1 | Validacion de tokens Bearer |
| Werkzeug | 3.1.3 | Capa WSGI y ProxyFix tras el balanceador |
| PostgreSQL | 16.15 | Motor de base de datos (AWS RDS) |
| Gunicorn | 23.0.0 | Servidor WSGI en produccion |

# 3. Diseno del API REST

## 3.1 POST /blacklists

Agrega un email a la lista negra global.

| Parametro | Tipo | Obligatorio | Reglas |
|---|---|---|---|

| Parametro | Tipo | Obligatorio | Reglas |
|---|---|---|---|
| email | String | Si | Formato valido, maximo 254 caracteres |
| app_uuid | String | Si | UUID valido de la app cliente |
| blocked_reason | String | No | Maximo 255 caracteres |

**Autorizacion:** `Authorization: Bearer <token>`

La **direccion IP** del solicitante y la **fecha/hora** de la solicitud se registran internamente y no se aceptan como parametros de entrada: un cuerpo que los incluya se rechaza con 400.

| Codigo | Significado |
|---|---|
| 201 | Email agregado (created: true) |
| 400 | Datos invalidos, con detalle en errors |
| 401 | Token ausente o invalido |
| 409 | El email ya estaba en la lista negra |

**Ejemplo de respuesta 201:**

```
{
  "created": true,
  "message": "El email fue agregado a la lista negra global.",
  "data": {
    "id": 1,
    "email": "spam@example.com",
    "app_uuid": "3f0c1b9a-6d2e-4a1b-9c8d-5e7f2a1b3c4d",
    "blocked_reason": "Reportado por abuso",
    "request_ip": "186.29.120.221",
    "created_at": "2026-10-17T16:40:09.285374+00:00"
  }
}
```

## 3.2 GET /blacklists/<email>

Consulta si un email esta en la lista negra global y el motivo por el que fue agregado. La consulta es insensible a mayusculas y minusculas.

Un email no listado responde **200 con blacklisted: false**, no 404: la consulta es exitosa y su resultado es negativo.

```
{ "email": "spam@example.com", "blacklisted": true, "blocked_reason": "Reportado por abuso" }
```

## 3.3 GET / — Health check

Verificacion de vida consumida por el balanceador de Elastic Beanstalk. Es publico y deliberadamente liviano: no consulta la base de datos, para que una latencia puntual del RDS no saque de servicio a las instancias.

### 3.4 Autorizacion

El enunciado permite definir el token Bearer de forma estatica o predefinida para toda la aplicacion. Ese token se lee de la variable de entorno STATIC_BEARER_TOKEN y **nunca esta escrito en el codigo fuente**. La comparacion se hace en tiempo constante (hmac.compare_digest) para no filtrar el token por diferencias de tiempo de respuesta.

## 4. Repositorio en GitHub

**URL:** [https://github.com/LorduyANDES/blacklist-microservice](https://github.com/LorduyANDES/blacklist-microservice)

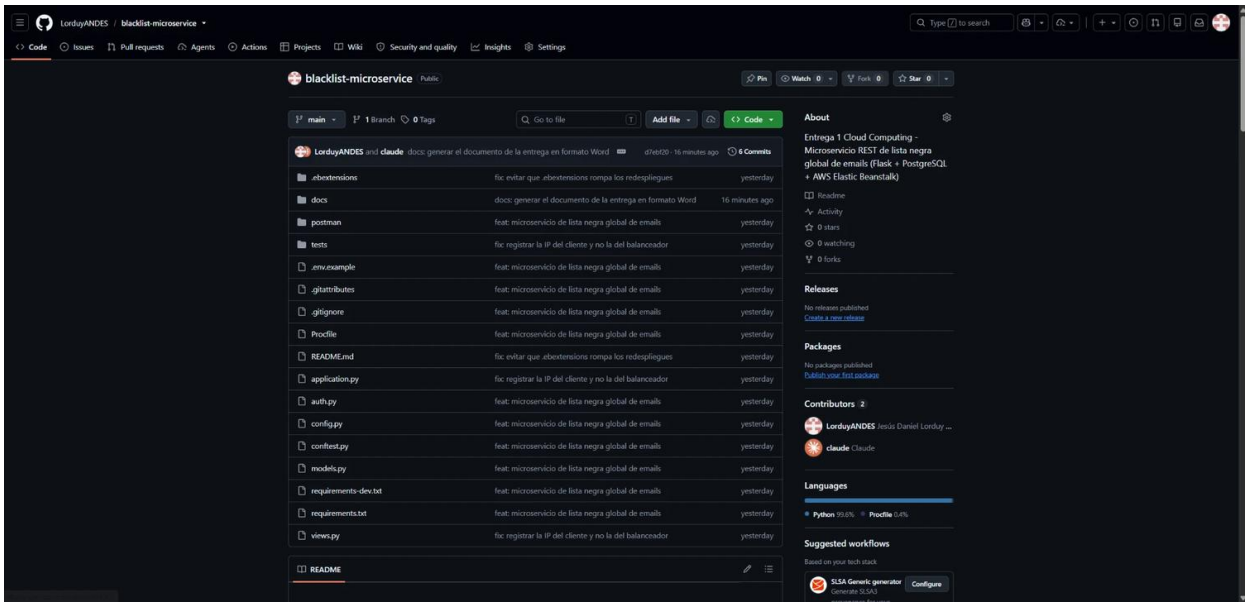



**Figura 1.** *Pagina principal del repositorio en GitHub, mostrando la lista de archivos y el README.*

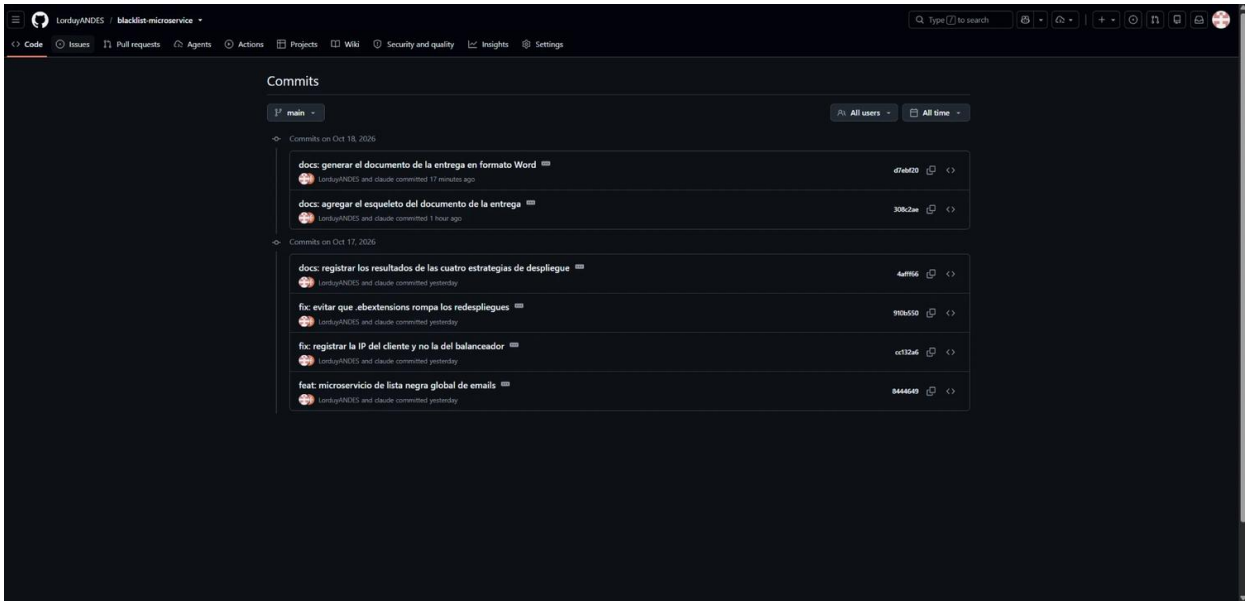



**Figura 2.** *Historial de commits (pestana Commits), evidenciando el versionamiento del proyecto.*

**Estructura del repositorio:**

```
application.py                  Punto de entrada WSGI exigido por Beanstalk
config.py                       Configuracion leida de variables de entorno
models.py                       Modelo SQLAlchemy y esquemas Marshmallow
auth.py                         Autorizacion por Bearer Token
views.py                        Recursos REST (Flask-RESTful)
conftest.py                     Fixtures de pruebas
tests/test_blacklists.py        25 pruebas del contrato del API
requirements.txt                Dependencias de ejecucion
Procfile                        Comando Gunicorn de arranque
.ebextensions/01_packages.config  Opciones de plataforma
postman/                        Coleccion y environment de Postman
docs/                           Documentacion de la entrega
```

**Nota sobre seguridad:** el archivo .env, que contiene el token Bearer y la contrasena del RDS, esta excluido mediante .gitignore y nunca fue versionado. Se verifico que ningun commit del historial contiene credenciales.

# 5. Configuracion de AWS

## 5.1 Configuracion de RDS

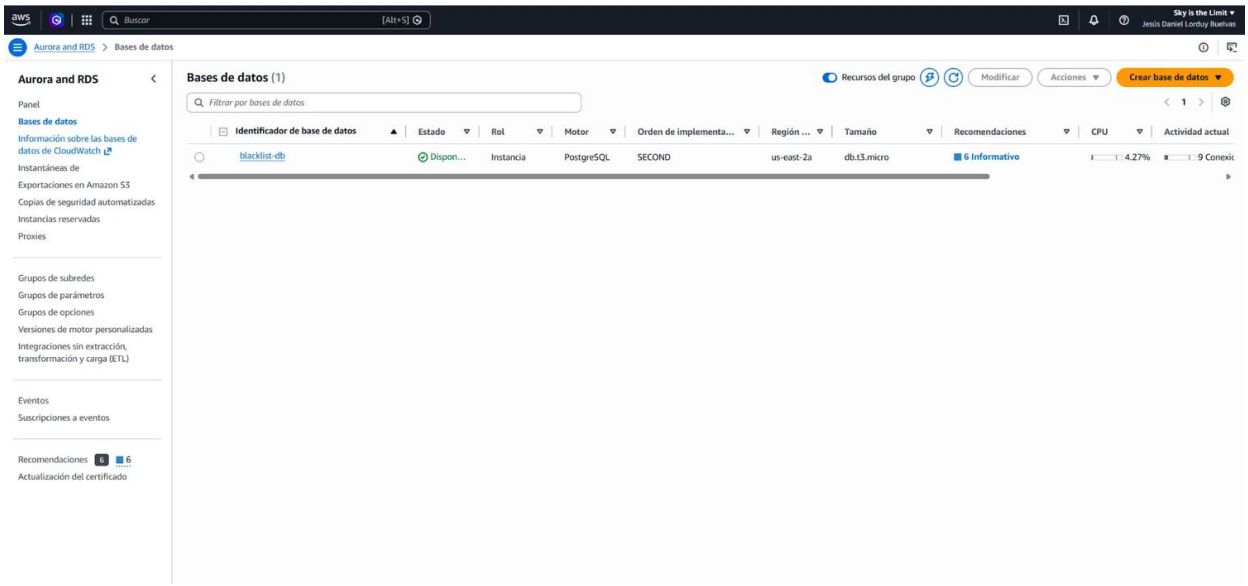



**Figura 3.** *Consola de RDS mostrando la instancia blacklist-db con estado Available.*

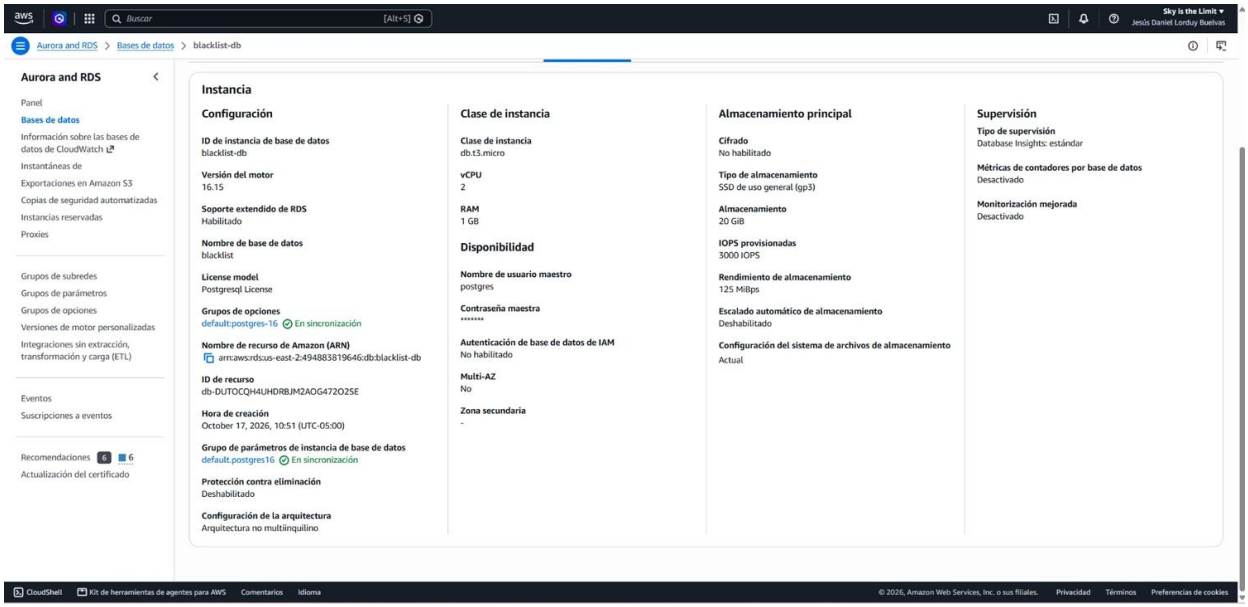



**Figura 4.** *Pestana Connectivity & security de la instancia, donde se vea Publicly accessible: No y el VPC security group asociado.*

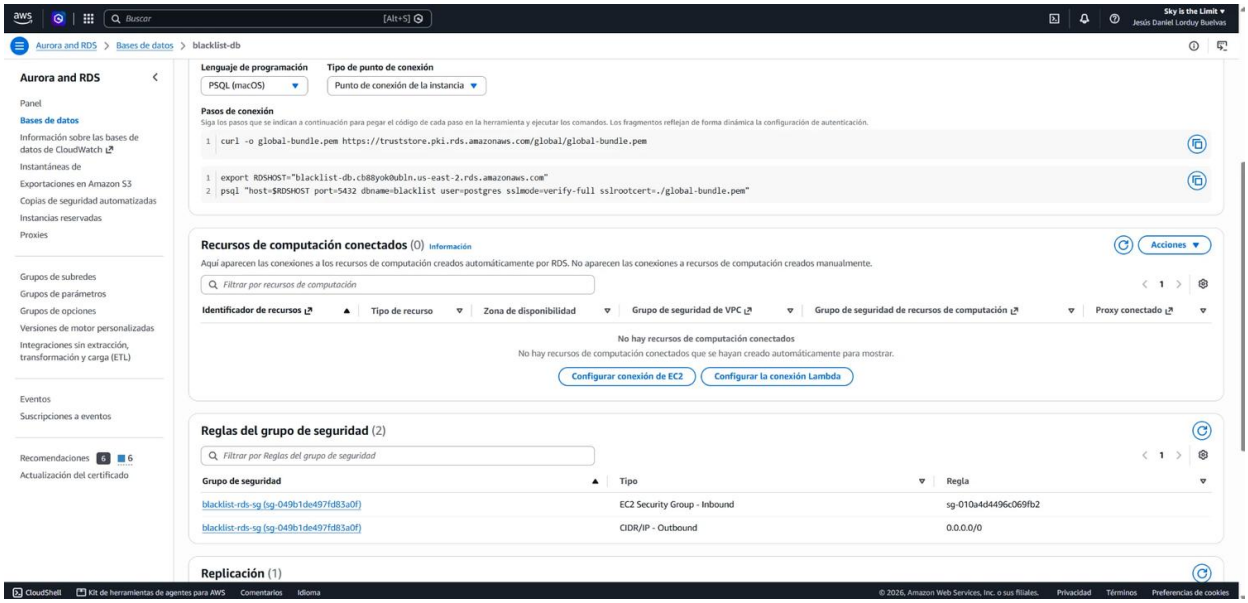



**Figura 5.** *Reglas de entrada del security group del RDS, mostrando que el puerto 5432 solo acepta trafico desde el security group de Beanstalk, NO desde 0.0.0.0/0.*

| Parametro | Valor |
|---|---|
| Identificador | blacklist-db |
| Motor | PostgreSQL 16.15 |
| Clase de instancia | db.t3.micro |
| Almacenamiento | 20 GB, tipo gp3 |
| Base de datos | blacklist |
| Usuario maestro | postgres |

| Parametro | Valor |
|---|---|
| Puerto | 5432 |
| Zona de disponibilidad | us-east-2a |
| Multi-AZ | No |
| Acceso publico | No |
| Endpoint | blacklist-db.XXXXXXXX.us-east-2.rds.amazonaws.com |

*El endpoint completo se omite de este documento por higiene de seguridad; el valor real esta en el archivo local .env y en las variables de entorno de Beanstalk.*

**Decision de diseno:** la base de datos no es accesible desde internet. Su grupo de seguridad permite el puerto 5432 unicamente desde el grupo de seguridad de las instancias EC2 del entorno de Beanstalk. Esto reduce la superficie de exposicion: aun si las credenciales se filtraran, la base de datos no seria alcanzable desde fuera de la VPC.

## 5.2 Configuracion del proyecto en AWS Elastic Beanstalk

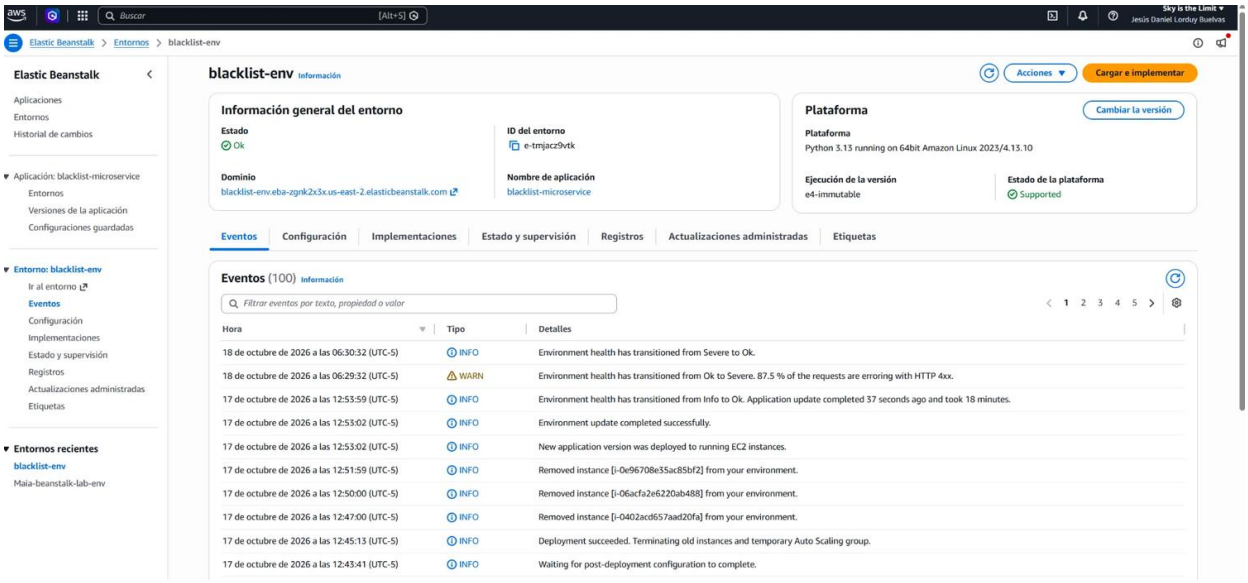



**Figura 6.** *Pagina del entorno blacklist-env mostrando Health: Ok (verde), el dominio y la version desplegada.*

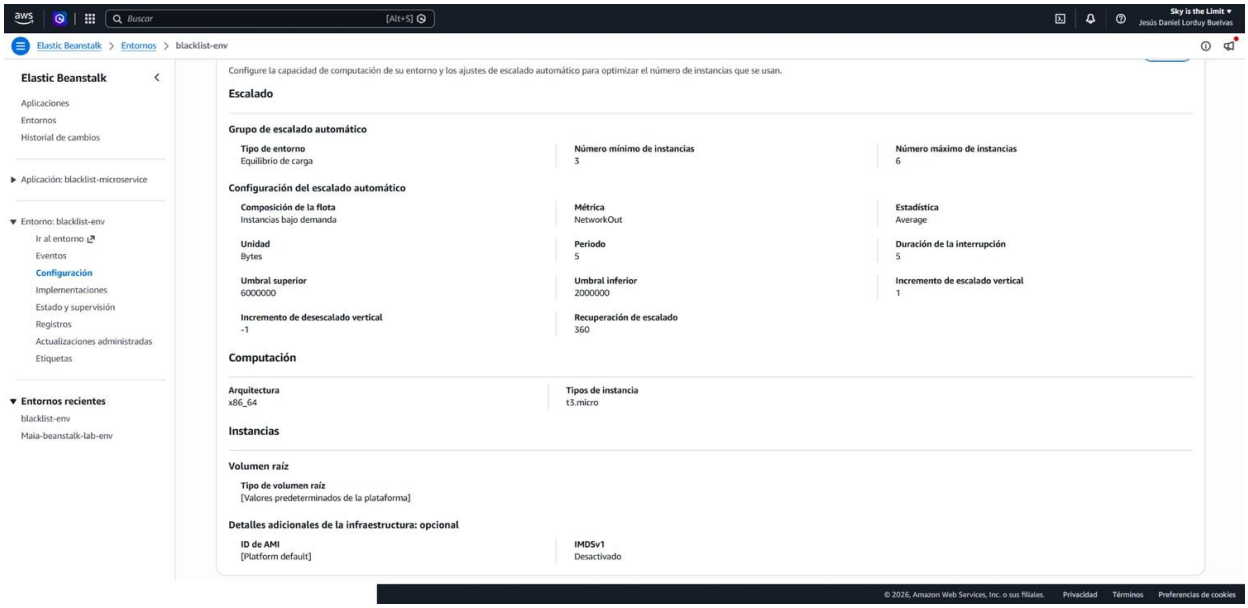



**Figura 7.** *Configuracion de capacidad del entorno: tipo con equilibrio de carga, grupo de autoescalado de minimo 3 y maximo 6 instancias, y tipo de instancia t3.micro.*

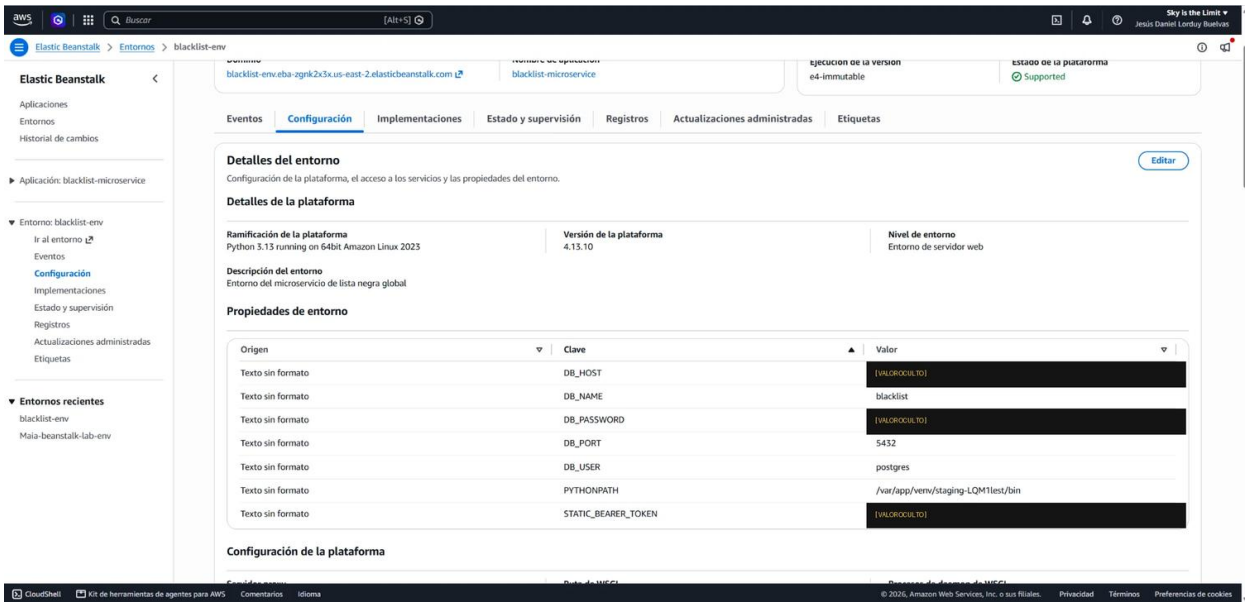



**Figura 8.** *Configuracion -> Propiedades del entorno, con las variables DB_USER, DB_HOST, DB_PORT, DB_NAME, DB_PASSWORD y STATIC_BEARER_TOKEN. IMPORTANTE: oculte los valores de DB_PASSWORD y STATIC_BEARER_TOKEN antes de capturar.*

| **Parametro** | **Valor** |
|---|---|
| Aplicacion | blacklist-microservice |
| Entorno | blacklist-env |
| ID del entorno | e-tmjacz9vtk |
| Region | us-east-2 (Ohio) |
| Plataforma | Python 3.13 sobre 64bit Amazon Linux 2023 / 4.13.10 |

| Parametro | Valor |
|---|---|
| Tipo de entorno | Con balanceador (Application Load Balancer) |
| Tipo de instancia | t3.micro |
| Autoescalado | Minimo 3, maximo 6 |
| URL | http://blacklist-env.eba-zgnk2x3x.us-east-2.elasticbeanstalk.com |

**Requisitos de la plataforma.** Elastic Beanstalk exige que el modulo principal se llame application.py y que la variable WSGI se llame application; por eso esos nombres no pueden cambiarse. El arranque se define en el Procfile:

```
web: gunicorn --bind 0.0.0.0:8000 --workers 3 --threads 2 --timeout 60 application:application
```

## 5.3 Configuracion de los health checks

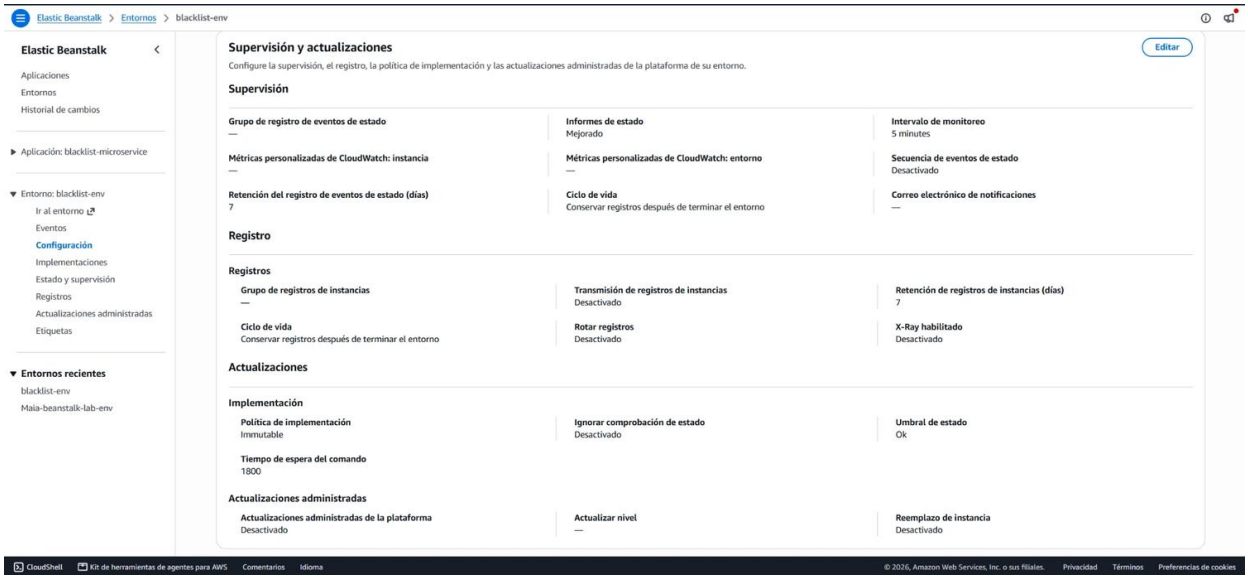



**Figura 9.** *Configuracion -> Supervision, mostrando la ruta del health check configurada en /.*

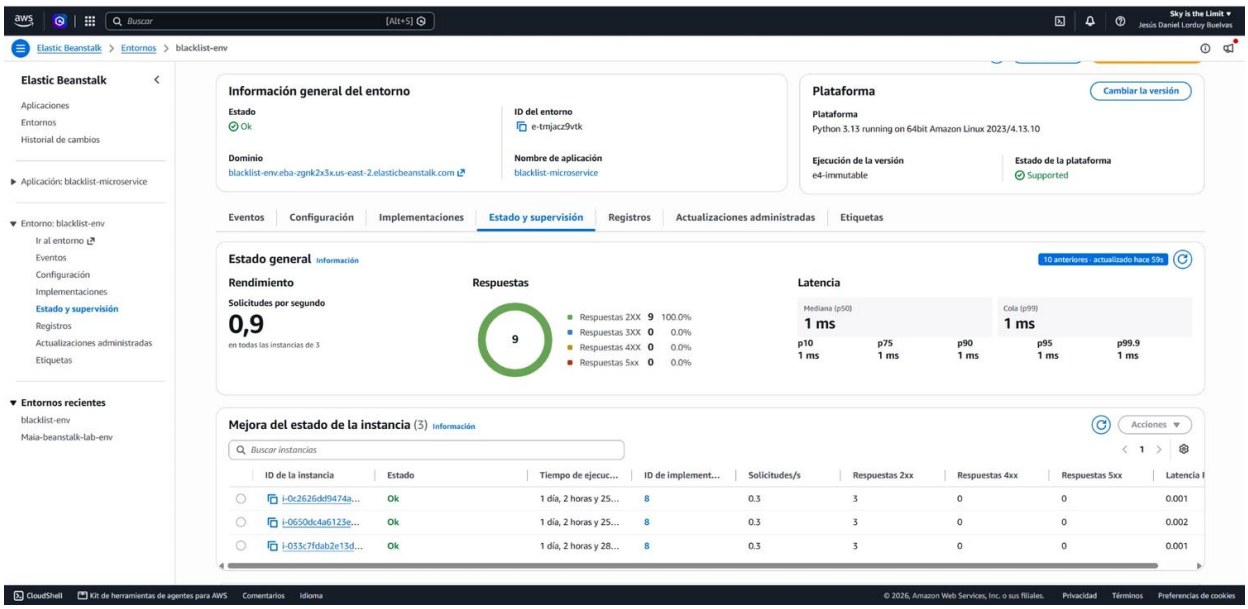



**Figura 10.** *Pestana Estado y supervision del entorno, mostrando las 3 instancias en estado Ok.*

| Parametro | Valor |
|---|---|
| Ruta del health check | / |
| Codigo HTTP esperado | 200 |
| Tipo de reporte de salud | Enhanced |

El endpoint / responde 200 con un JSON de estado. Se eligio deliberadamente que **no consulte la base de datos**: si el RDS tuviera una latencia puntual, un health check dependiente de la base sacaria de servicio a instancias que en realidad estan sanas, provocando un reemplazo en cascada.

Esa ruta se fija ademas desde el repositorio, en .ebextensions/01_packages.config, de modo que la configuracion viaja con el codigo y no depende de ajustes manuales en la consola.

## 6. Documentacion y escenarios de prueba en Postman

**URL de la documentacion publicada:**
https://documenter.getpostman.com/view/59043927/2sBYHSy11e

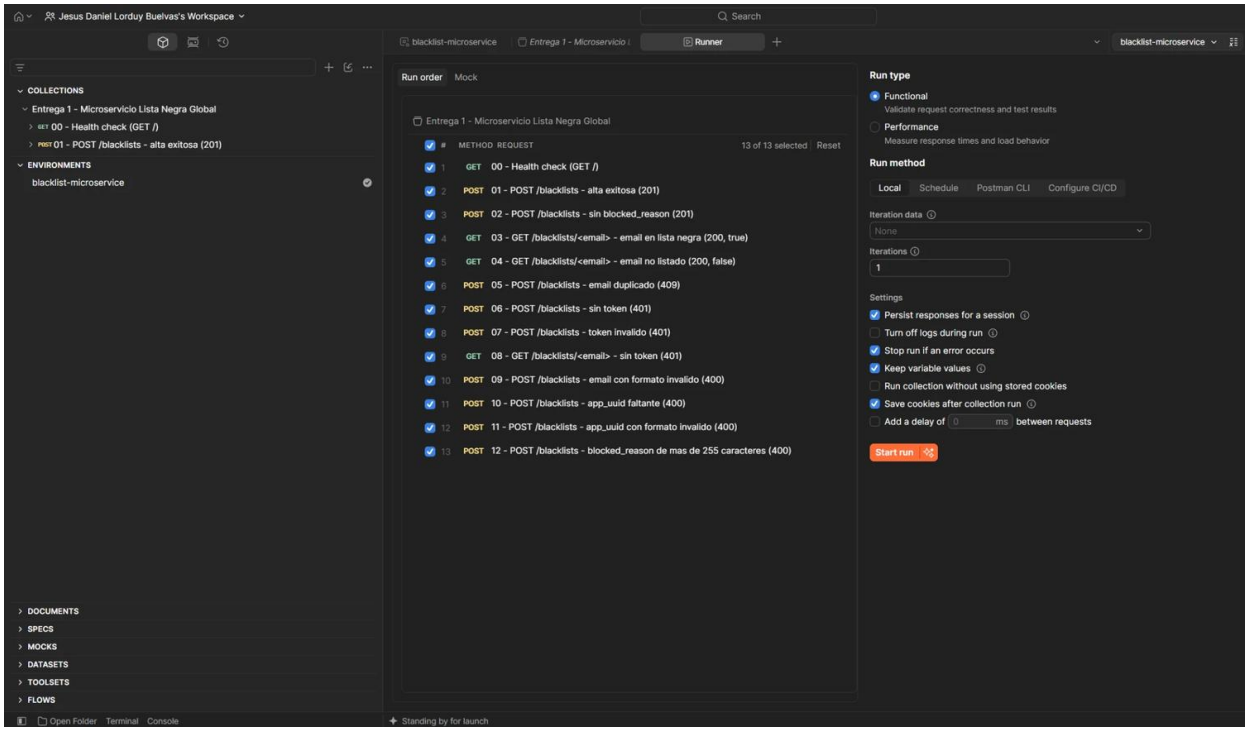



**Figura 11.** *Coleccion de Postman abierta, mostrando la lista de escenarios.*

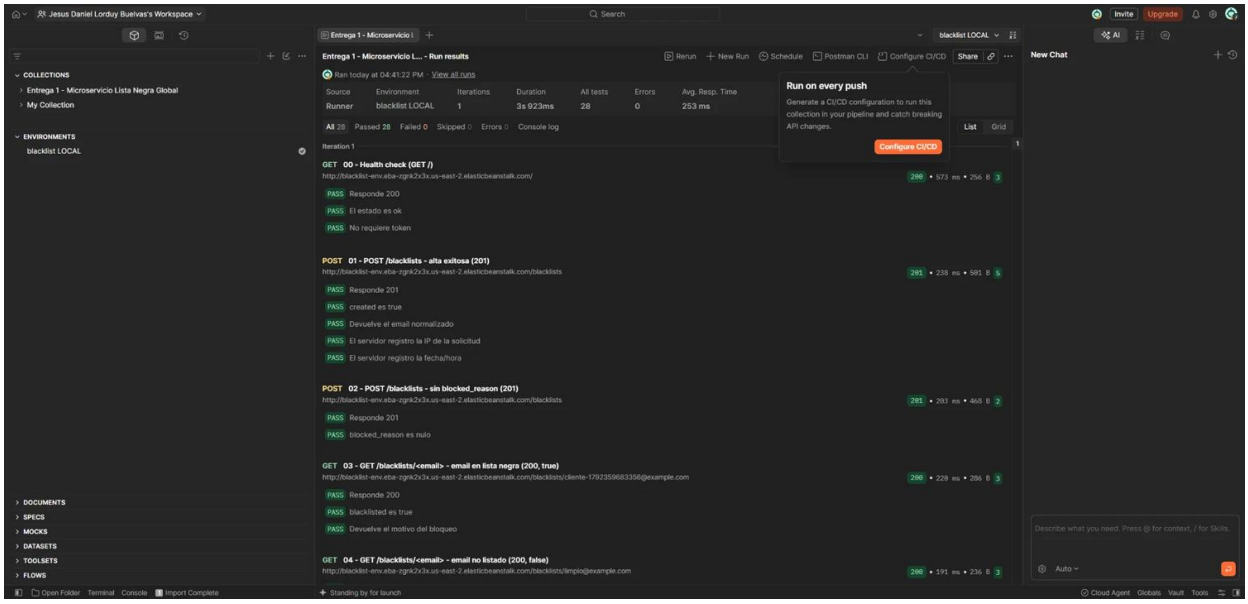



**Figura 12.** *Ejecucion del Collection Runner con los escenarios en verde.*

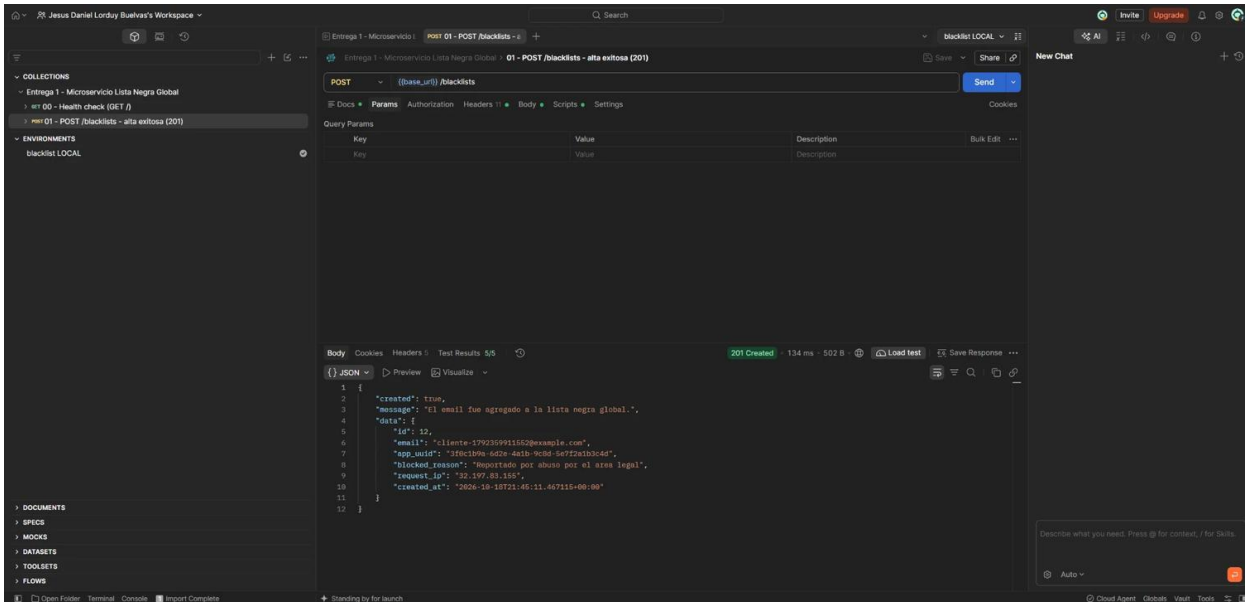



**Figura 13.** *Ejemplo de solicitud y respuesta de POST /blacklists (201).*

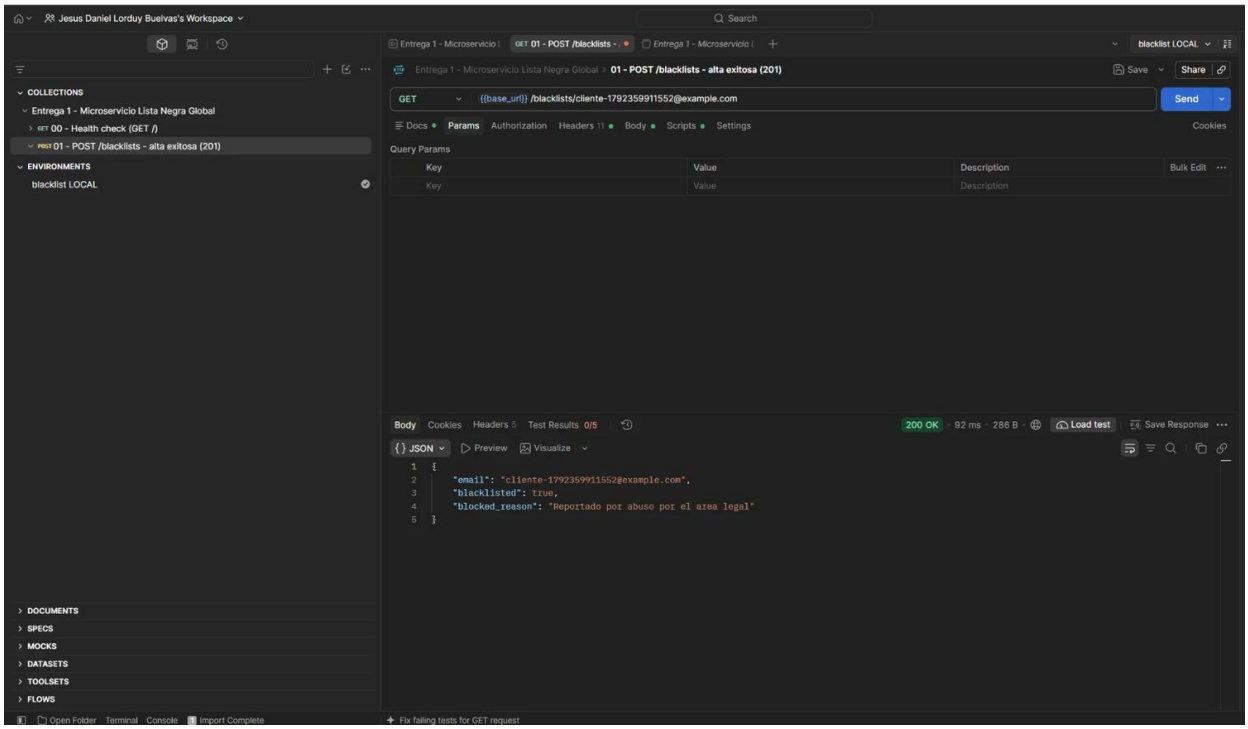



**Figura 14.** *Ejemplo de solicitud y respuesta de GET /blacklists/<email>.*

La coleccion y el environment estan versionados en el repositorio, en la carpeta postman/. La coleccion es autosuficiente: puede ejecutarse de corrido con el Collection Runner, porque los escenarios de consulta y de duplicado reutilizan el email que crea el escenario 01.

| # | Escenario | Resultado esperado |
|---|---|---|
| 00 | Health check GET / | 200 |

| # | Escenario | Resultado esperado |
|---|---|---|
| 01 | Alta exitosa | 201 |
| 02 | Alta sin blocked_reason | 201 |
| 03 | Consulta de email en lista negra | 200, blacklisted: true |
| 04 | Consulta de email no listado | 200, blacklisted: false |
| 05 | Alta de email duplicado | 409 |
| 06 | POST sin token | 401 |
| 07 | POST con token invalido | 401 |
| 08 | GET sin token | 401 |
| 09 | Email con formato invalido | 400 |
| 10 | app_uuid faltante | 400 |
| 11 | app_uuid con formato invalido | 400 |
| 12 | blocked_reason de mas de 255 caracteres | 400 |

*El environment se exporta con bearer_token vacio a proposito, para no versionar el secreto. Debe completarse al importarlo.*

**Pruebas automatizadas.** Adicionalmente, el repositorio incluye 25 pruebas automatizadas del contrato del API (pytest), que cubren autorizacion, validacion de cada campo, duplicados, normalizacion de emails, registro interno de IP y fecha/hora, e imposibilidad de suplantar la IP del cliente.

**Salida de la ejecucion de las pruebas:**

```
============================= test session starts =============================
platform win32 -- Python 3.13.14, pytest-8.3.4, pluggy-1.7.0
rootdir: C:/Users/danie/Desktop/blacklist-microservice
collected 25 items

tests/test_blacklists.py .........................              [100%]

============================= 25 passed in 39.16s =============================
```

# 7. Estrategias de despliegue

Se ejecutaron cuatro estrategias distintas sobre el entorno blacklist-env, todas sobre un grupo de autoescalado de minimo 3 y maximo 6 instancias.

## 7.1 Resultados comparados

| Estrategia | Tiempo total | Antes | Pico | Despues | Nuevas | Iniciales o nuevas |
|---|---|---|---|---|---|---|
| All at once | 0 m 28 s | 3 | 3 | 3 | 0 | Sobre las iniciales |
| Rolling (33 %) | 8 m 14 s | 3 | 3 | 3 | 0 | Sobre las iniciales |
| Rolling with additional batch (33 %) | 9 m 27 s | 3 | 4 | 3 | 1 | Mayoria iniciales, 1 nueva |
| Immutable | 18 m 42 s | 3 | 6 | 3 | 3 | Todas nuevas |

## 7.2 Como se valido que cada despliegue quedo en ejecucion

Para las cuatro estrategias se aplico la misma verificacion, combinando el estado reportado por AWS con pruebas funcionales reales contra la URL publica:

- Estado del entorno: se espero a que Status volviera a Ready y se registro Health, HealthStatus y VersionLabel, para confirmar que la version efectivamente desplegada era la nueva y no la anterior.
- Health check HTTP: GET / debia responder 200.
- Escritura real en base de datos: POST /blacklists con un email unico debia responder 201, lo que prueba que la aplicacion no solo arranco sino que conserva conectividad con el RDS.
- Lectura real: GET /blacklists/<email> sobre ese email debia responder 200 con blacklisted: true.
- Registro de eventos: se capturo el log de eventos para la ventana exacta del despliegue.

Las cuatro estrategias superaron los cuatro puntos.

## 7.3 All at once — 0 m 28 s

**Registro de eventos del despliegue, capturado durante su ejecucion:**

```
17:11:37  Environment update is starting.
17:11:42  Deploying new version to instance(s).
17:11:46  Instance deployment used the commands in your 'Procfile' to
          initiate startup of your application.
17:11:55  Instance deployment completed successfully.
17:12:00  New application version was deployed to running EC2 instances.
17:12:00  Environment update completed successfully.
17:12:01  Application update in progress on 3 instances.
```

Las tres instancias se actualizan en una sola operacion, sin lotes: del inicio al fin transcurren 23 segundos de reloj de AWS. Este despliegue quedo fuera de la ventana de 100 eventos que conserva la consola, por lo que se documenta mediante el registro capturado en el momento de su ejecucion.

No se dispone de una captura de la vista de instancias durante este despliegue: al durar 28 segundos, la ventana de observacion se cerro antes de poder registrarla. El efecto queda documentado por el registro de eventos anterior, que confirma la actualizacion simultanea de las tres instancias.

- Instancias utilizadas: 3 (las mismas antes y despues).
- Sobre instancias iniciales o nuevas: sobre las iniciales, 0 nuevas.

**Hallazgos:** es con diferencia la mas rapida, unas 18 veces mas rapida que Rolling. Actualiza las tres instancias a la vez y no espera a que el balanceador vuelva a registrarlas como sanas antes de dar por terminado el despliegue. Ese es exactamente su costo: durante esos segundos ninguna instancia sirve la version estable, de modo que hay una ventana de indisponibilidad. Aceptable en desarrollo, desaconsejable en produccion.

## 7.4 Rolling — 8 m 14 s

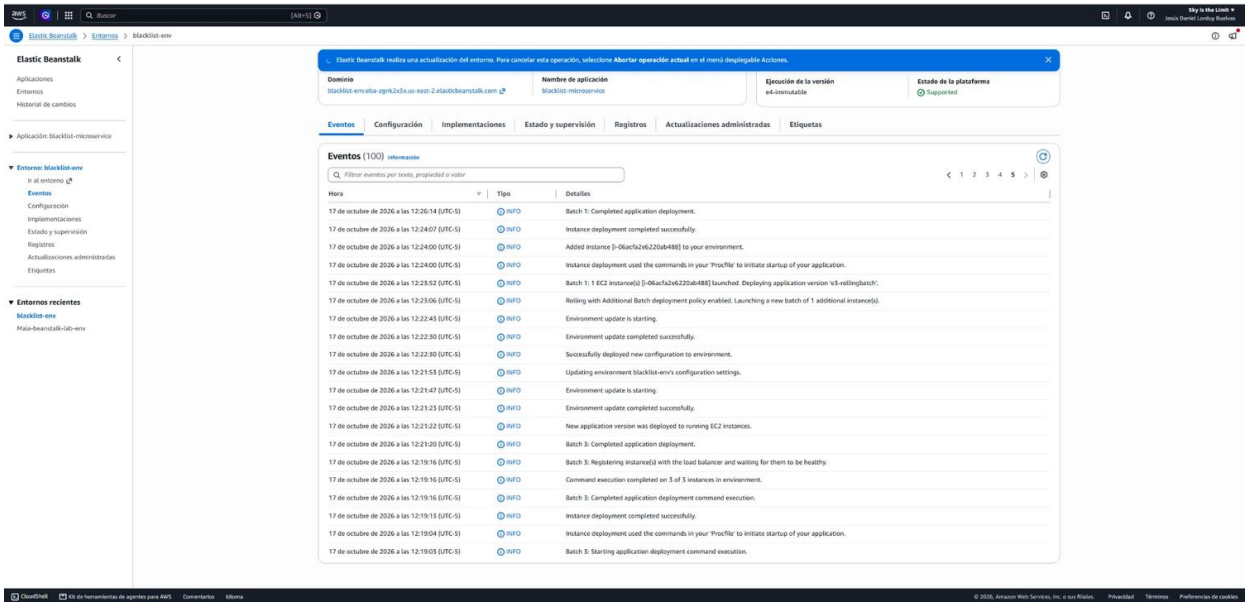



**Figura 15.** *Eventos mostrando los lotes (Batch 1, Batch 2, Batch 3).*

- Instancias utilizadas: 3 (las mismas antes y despues). Lotes del 33 %.
- Sobre instancias iniciales o nuevas: sobre las iniciales, 0 nuevas.

**Hallazgos:** despliega en tres lotes de una instancia; en cada uno espera a que la instancia se registre de nuevo en el balanceador y quede sana. Eso explica la diferencia de tiempo frente a All at once. No hay interrupcion del servicio, pero si capacidad reducida: mientras un lote se actualiza solo quedan dos de tres instancias atendiendo. Con carga cercana al limite, esa perdida de un tercio de la capacidad puede degradar los tiempos de respuesta.

## 7.5 Rolling with additional batch — 9 m 27 s

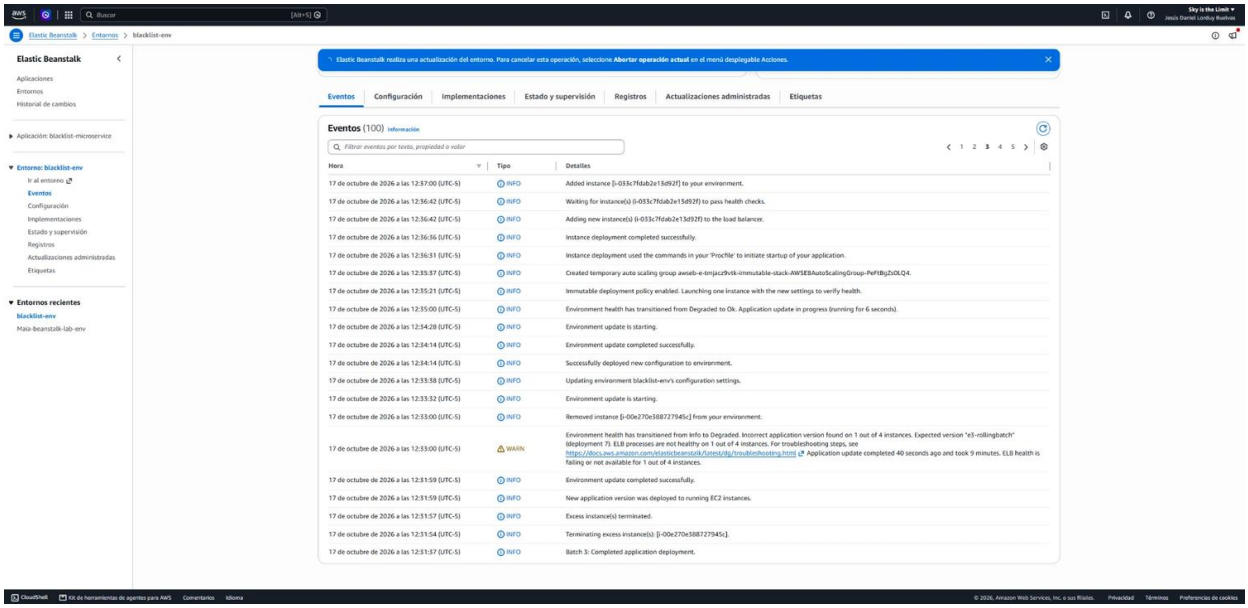


**Figura 16.** *Eventos mostrando el lote adicional y el mensaje Terminating excess instance(s).*

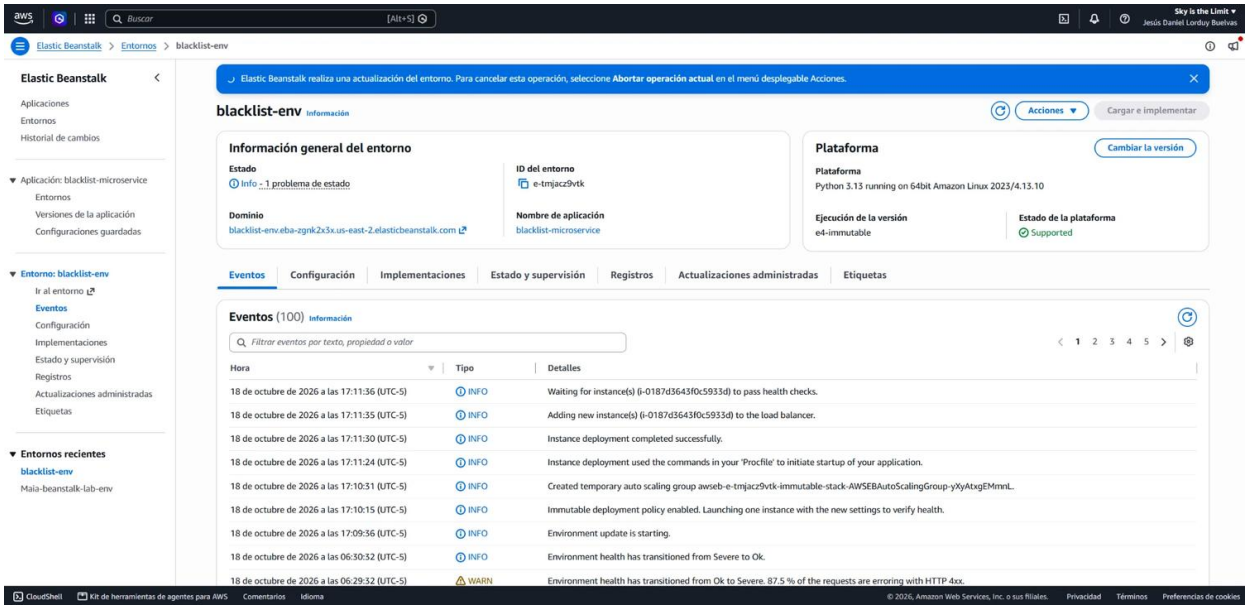


**Figura 17.** *Inicio de un despliegue Immutable: la politica se habilita, se crea el grupo de autoescalado temporal y la primera instancia nueva se somete a los health checks antes de lanzar el resto.*

- Instancias utilizadas: 3 iniciales + 1 adicional = 4 en el pico; vuelve a 3.
- Sobre instancias iniciales o nuevas: mayoria iniciales, 1 nueva.

**Hallazgos:** solo 1 m 13 s mas lenta que Rolling, y resuelve justamente su punto debil. Antes de empezar lanza un lote adicional y al terminar elimina la sobrante. El log lo muestra de forma explicita:

```
Batch 3: Completed application deployment.
Terminating excess instance(s): [i-00e270e388727945c].
Excess instance(s) terminated.
```

Gracias a ese lote extra la capacidad nunca baja de las tres instancias originales. Es el mejor equilibrio entre tiempo y disponibilidad: por poco mas de un minuto se elimina la degradacion de capacidad.

Observacion: al finalizar, el entorno quedo momentaneamente en estado Degraded mientras la instancia excedente terminaba de darse de baja, aunque el servicio respondio correctamente durante todo el proceso.

## 7.6 Immutable — 18 m 42 s

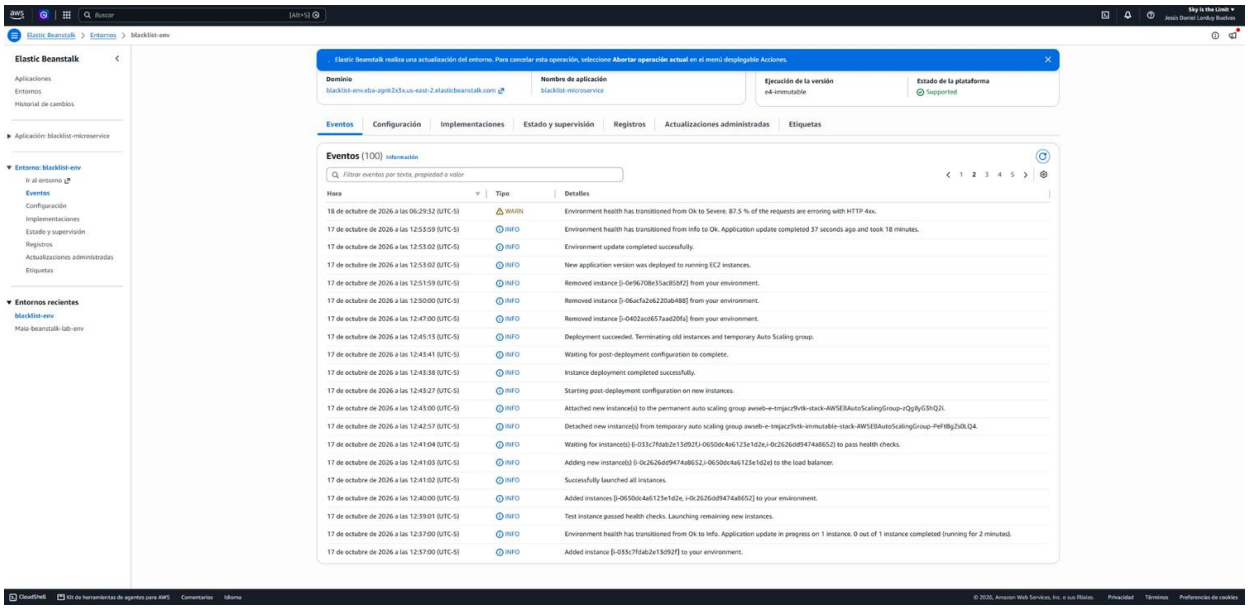



**Figura 18.** *Eventos mostrando Created temporary auto scaling group y Deployment succeeded. Terminating old instances and temporary Auto Scaling group.*

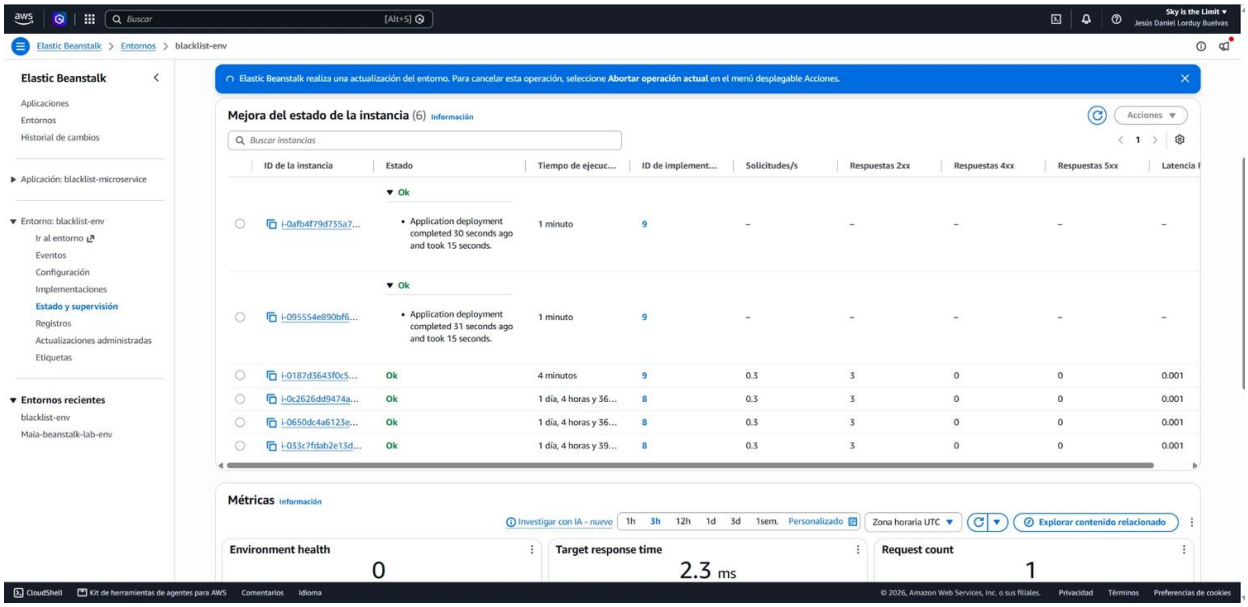



**Figura 19.** *Pestana Estado y supervision durante el despliegue, con 6 instancias simultaneas. Es la evidencia visual mas clara de las cuatro.*

- Instancias utilizadas: 3 iniciales + 3 nuevas = 6 en el pico; termina con 3.
- Sobre instancias iniciales o nuevas: sobre instancias completamente nuevas; las 3 iniciales fueron reemplazadas.

**Hallazgos:** la mas lenta y la unica que reemplaza el 100 % de las instancias. Crea un grupo de autoescalado temporal paralelo, levanta primero una instancia para verificar que la nueva version arranca sana, y solo entonces completa el resto:

```
Immutable deployment policy enabled. Launching one instance with the new
settings to verify health.
Created temporary auto scaling group awseb-e-tmjacz9vtk-immutable-stack-...
Detached new instance(s) from temporary auto scaling group ...
Deployment succeeded. Terminating old instances and temporary Auto Scaling group.
```

El entorno llega a 6 instancias simultaneas, que es precisamente el maximo configurado: **con un maximo menor que el doble del minimo, esta estrategia no cabria**. Eso justifica la recomendacion del enunciado de usar un grupo de 3 a 6.

Es la mas segura de las cuatro: si la nueva version no arranca, las instancias viejas nunca se tocan y la reversion es inmediata. A cambio, consume el doble de instancias y el triple de tiempo.

## 7.7 Conclusion sobre las estrategias

No hay una estrategia mejor en abstracto; la eleccion depende de que se quiera proteger:

- Velocidad por encima de todo, sin importar la caida: All at once.
- Sin caida, aceptando capacidad reducida: Rolling.
- Sin caida y sin perder capacidad, por poco mas de tiempo: Rolling with additional batch. Para este servicio es la opcion mas equilibrada.
- Maxima seguridad y reversion instantanea: Immutable, asumiendo el doble de instancias y el triple de tiempo.

## 8. Hallazgo adicional: un .ebextensions puede romper los redespliegues

Durante las pruebas se encontro un fallo que merece registrarse, porque su sintoma no apunta a su causa.

El archivo .ebextensions/01_packages.config instalaba los paquetes de sistema libpq-devel y gcc. La creacion inicial del entorno funciono sin problemas, pero todos los redespliegues posteriores fallaban con un mensaje generico:

```
[Instance: i-xxxx] Command failed on instance.
An unexpected error has occurred [ErrorCode: 0000000001].
Command execution completed on all instances.
Summary: [Successful: 0, Failed: 2, TimedOut: 1].
Failed to deploy application.
```

Ese mensaje no indica que fallo. La causa solo se encontro leyendo eb-engine.log en la instancia, donde el registro se corta exactamente en la fase que ejecuta los .ebextensions:

```
[INFO] Executing instruction: PreBuildEbExtension
[INFO] Running command: /opt/aws/bin/cfn-init ... Infra-EmbeddedPreBuild
      (sin mas salida; 21 segundos despues, "Command failed")
```

Agrava el problema que las instancias quedan bloqueadas en estado "Performing application deployment" indefinidamente y hay que reemplazarlas manualmente.

Esos paquetes nunca fueron necesarios: el proyecto usa psycopg2-binary, que se distribuye como rueda precompilada y no requiere compilador ni cabeceras de PostgreSQL. Se reemplazo el bloque packages por option_settings, y a partir de ahi los cuatro despliegues se ejecutaron sin incidentes.

**Leccion:** un .ebextensions que funciona al crear el entorno no esta validado hasta que se ha probado un redespliegue sobre instancias ya existentes.

## 9. Segundo hallazgo: la IP registrada detras del balanceador

El enunciado exige guardar la direccion IP desde donde se hace la solicitud. La primera version registraba 172.31.35.99, una direccion privada de la VPC, no la del cliente.

La causa es que en Elastic Beanstalk la cadena de proxies es cliente -> ALB -> nginx -> gunicorn, y cada proxy agrega un salto a la cabecera X-Forwarded-For. Confiando en un solo salto se obtenia la IP del balanceador.

La correccion consistio en confiar en dos saltos (ProxyFix con x_for=2), lo que se verifico en produccion comparando la IP registrada con la IP publica real del cliente:

```
Mi IP real:    186.29.120.221
IP registrada: 186.29.120.221   <- correcto
```

Exigir dos saltos tiene ademas un efecto de seguridad: **un cliente no puede suplantar su IP** enviando su propio X-Forwarded-For, porque no alcanza los saltos de confianza requeridos. Ese comportamiento quedo cubierto con una prueba automatizada.

## 10. Conclusiones

El desarrollo y despliegue de este microservicio deja cinco conclusiones que surgen de lo observado durante el trabajo, no de consideraciones teoricas previas.

### Un PaaS elimina la administracion de servidores, no la de la configuracion

Ninguno de los dos fallos encontrados estuvo en el codigo de la aplicacion. El primero fue un archivo .ebextensions que instalaba paquetes innecesarios y rompia todos los redespliegues; el segundo, una cadena de proxies mal contada que hacia registrar la IP del balanceador en lugar de la del cliente. Elastic Beanstalk se encargo de aprovisionar instancias, balanceador y escalado, pero la configuracion de la plataforma siguio siendo responsabilidad del equipo, y es justamente donde aparecieron los problemas.

### La estrategia de despliegue es una decision de negocio

Las mediciones muestran un rango de 0 m 28 s a 18 m 42 s para desplegar exactamente el mismo codigo. La diferencia no es tecnica sino de prioridades: All at once cambia disponibilidad por velocidad, Immutable cambia tiempo y recursos por seguridad, y Rolling with additional batch ofrece el mejor equilibrio al evitar la perdida de capacidad por poco mas de un minuto adicional. Quien elige la politica esta decidiendo cuanta interrupcion tolera el negocio.

### El dimensionamiento del grupo de autoescalado condiciona que estrategias son posibles

Immutable llego a seis instancias simultaneas, exactamente el maximo configurado. Con un maximo inferior al doble del minimo, esa estrategia simplemente no cabe. La recomendacion del enunciado de usar un grupo de 3 a 6 no es arbitraria: es el requisito para poder ejecutar las cuatro politicas.

### Separar las credenciales del codigo permitio publicar el repositorio sin exponer secretos

Toda la configuracion sensible viaja por variables de entorno y el archivo .env quedo excluido del control de versiones desde el primer commit. Eso permitio mantener el repositorio publico y, al verificar el historial completo, confirmar que ninguna version contiene el token ni la contrasena de la base de datos. La misma disciplina obligo a censurar los valores en la captura de propiedades del entorno antes de incluirla en este documento.

### Diagnosticar en la nube exige saber donde viven los registros

El fallo de los redespliegues se manifestaba como "Command failed on instance. An unexpected error has occurred [ErrorCode: 0000000001]", un mensaje que no indica ninguna causa. Solo la lectura de eb-engine.log dentro de la instancia revelo que la ejecucion se detenia en la fase PreBuildEbExtension. La observabilidad de la plataforma resulto tan necesaria como la de la propia aplicacion.

## Anexo: como reproducir el despliegue

- Crear la instancia RDS PostgreSQL en la misma VPC y region del entorno.
- Configurar el grupo de seguridad del RDS para aceptar el puerto 5432 unicamente desde el grupo de seguridad de las instancias de Beanstalk.
- Crear la aplicacion y el entorno en Elastic Beanstalk con plataforma Python.
- Configurar las variables de entorno DB_USER, DB_PASSWORD, DB_HOST, DB_PORT, DB_NAME y STATIC_BEARER_TOKEN.
- Desplegar el paquete de codigo fuente. La tabla blacklist_entries se crea automaticamente en el primer arranque.